
Stop Paying 3s for Repeated LLM Queries

- 1 Why exact-match Redis caching fails in AI apps
- 2 A practical guide to semantic caching with vector search

The Core Problem

- 1 'Reset password' vs 'Forgot my credentials' = 0% cache hit rate with string matching
- 2 LLM API calls burn 3,000ms and real money for essentially identical user intents

The Architecture Fix

- 1 Convert user queries to vector embeddings with a fast model (10ms)
- 2 Perform vector search in Redis; return cached reply if cosine similarity > 0.90
- 3 On cache miss, call LLM once and store the new embedding vector

The Implementation Pattern

- 1 Query Redis vector index using similarity range search before hitting provider APIs

```
main.py

query_vec = encoder.encode(query)
cached = redis_client.ft().search(
    "(@embedding:[VECTOR_RANGE 0.9 $vec])",
    {"vec": query_vec.tobytes()}
)
if cached:
    return cached[0].response
```

The Real-World Impact

- 1 Response latency drops from 3000ms down to 50ms on cache hits
- 2 Cuts total LLM API bills by 60-80% on customer support workloads

Key Rule of Thumb

- 1 Keep cosine threshold above 0.90 to avoid returning stale or irrelevant responses
- 2 Follow for more pragmatic AI backend system design patterns

